

Operating Systems

Name: Ganga Gowri P

Roll Number: 241801070

Best Fit

```
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int blockSize[m];
    int original[m];
    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        original[i] = blockSize[i];
    }
    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }
    for(int i = 0; i < n; i++) {
        int bestIndex = -1;
        for(int j = 0; j < m; j++) {
            if(blockSize[j] >= processSize[i]) {
                if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
                    bestIndex = j;
                }
            }
        }
        if(bestIndex != -1) {
            int fragment = blockSize[bestIndex] - processSize[i];
            printf("Process %d of size %d is allocated to Block %d of size %d\n",
                i + 1, processSize[i], bestIndex + 1,
                original[bestIndex], fragment);
            blockSize[bestIndex] -= processSize[i];
        }
        else {
            printf("Process %d of size %d is not allocated\n",
                i + 1, processSize[i]);
        }
    }
    return 0;
}
```

FCFS

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
    }
    CT[0] = AT[0] + BT[0];
    for(i = 1; i < n; i++) {
        if(CT[i-1] < AT[i])
            CT[i] = AT[i] + BT[i];
        else
            CT[i] = CT[i-1] + BT[i];
    }
    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f\n", totalWT/n);
    return 0;
}
```

SJF

```
#include <stdio.h>
int main() {
    int n, i, j, time = 0, completed = 0, min;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n], visited[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
        visited[i] = 0;
    }
    while(completed < n) {
        min = -1;
        for(i = 0; i < n; i++) {
            if(AT[i] <= time && visited[i] == 0) {
                if(min == -1 || BT[i] < BT[min]) {
                    min = i;
                }
            }
        }
        if(min == -1) {
            time++;
        }
        else {
            time += BT[min];
            CT[min] = time;
            visited[min] = 1;
            completed++;
        }
    }
    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
    printf("Average Waiting Time = %.2f\n", totalWT/n);
    return 0;
}
```

Priority

```
#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0, min;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int AT[n], BT[n], P[n], CT[n], TAT[n], WT[n], visited[n];
    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: \n", i+1);
        scanf("%d %d %d", &AT[i], &BT[i], &P[i]);
        visited[i] = 0;
    }
    while(completed < n) {
        min = -1;
        for(i = 0; i < n; i++) {
            if(AT[i] <= time && visited[i] == 0) {
                if(min == -1 || P[i] < P[min]) {
                    min = i;
                }
            }
        }
        if(min == -1) {
            time++;
        }
        else {
            time += BT[min];
            CT[min] = time;
            visited[min] = 1;
            completed++;
        }
    }
    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
    printf("Average Waiting Time = %.2f\n", totalWT/n);
    return 0;
}
```

Round Robin

```
#include <stdio.h>
int main() {
    int n, tq, i, time = 0, completed = 0;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);
    int AT[n], BT[n], RT[n], CT[n], TAT[n], WT[n];
    for(i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d: \n", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
        RT[i] = BT[i];
    }
    while(completed < n) {
        int done = 1;
        for(i = 0; i < n; i++) {
            if(RT[i] > 0 && AT[i] <= time) {
                done = 0;
                if(RT[i] > tq) {
                    time += tq;
                    RT[i] -= tq;
                }
                else {
                    time += RT[i];
                    CT[i] = time;
                    RT[i] = 0;
                    completed++;
                }
            }
        }
        if(done)
            time++;
    }
    float totalTAT = 0, totalWT = 0;
    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }
    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f\n", totalWT/n);
    return 0;
}
```

Banker's Algorithm

```
#include <stdio.h>
int main() {
    int p, r, i, j, k;
    printf("Enter number of processes: \n");
    scanf("%d", &p);
    printf("Enter number of resource types: \n");
    scanf("%d", &r);
    int alloc[p][r], max[p][r], need[p][r];
    int avail[r], work[r];
    int finish[p], safeSeq[p];
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available Resources: \n");
    for(i = 0; i < r; i++) {
        scanf("%d", &avail[i]);
        work[i] = avail[i];
    }
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    for(i = 0; i < p; i++)
        finish[i] = 0;
    int count = 0;
    while(count < p) {
        int found = 0;
        for(i = 0; i < p; i++) {
            if(finish[i] == 0) {
                int possible = 1;
                for(j = 0; j < r; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }
                if(possible) {
                    for(k = 0; k < r; k++)
                        work[k] += alloc[i][k];
                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }
    if(count == p) {
```

```
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
        for(i = 0; i < p; i++)
            printf("P%d ", safeSeq[i]);
    }
    else {
        printf("System is NOT in a safe state (Deadlock may occur).\n");
    }
    return 0;
}
```

Producer Consumer

```
#include <stdio.h>
#include <stdlib.h>
int mutex = 1, full = 0, empty = 5, x = 0;
int wait(int s)
{
    return --s;
}
int signal(int s)
{
    return ++s;
}
void producer()
{
    mutex = wait(mutex);
    empty = wait(empty);
    full = signal(full);
    x++;
    printf("\nProducer produces item %d", x);
    mutex = signal(mutex);
}
void consumer()
{
    mutex = wait(mutex);
    full = wait(full);
    empty = signal(empty);
    printf("\nConsumer consumes item %d", x);
    x--;
    mutex = signal(mutex);
}
int main()
{
    int n;
    printf("\n----- Producer Consumer Problem -----");
    printf("\n1. Producer");
    printf("\n2. Consumer");
    printf("\n3. Exit");
    while(1)
    {
        printf("\nEnter your choice: ");
        scanf("%d", &n);
        switch(n)
        {
            case 1:
                if(mutex==1 && empty!=0)
                {
                    producer();
                }
                else
                {
                    printf("\nBuffer is full!! Producer cannot produce.");
                }
                break;
            case 2:
```

```
        if(mutex==1 && full!=0)
        {
            consumer();
        }
        else
        {
            printf("\nBuffer is empty!! Consumer cannot consume.");
        }
        break;
    case 3:
        printf("\nExiting program...");
        printf("\nFinal buffer items produced: %d\n", x);
        exit(0);
    default:
        printf("\nInvalid choice. Try again.");
    }
}
return 0;
}
```

First Fit

```
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int blockSize[m];
    int originalBlockSize[m];
    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        originalBlockSize[i] = blockSize[i];
    }
    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }
    for(int i = 0; i < n; i++) {
        int allocated = 0;
        for(int j = 0; j < m; j++) {
            if(blockSize[j] >= processSize[i]) {
                int fragment = blockSize[j] - processSize[i];
                printf("Process %d of size %d is allocated to Block %d of\n",
size %d with Fragment %d\n",
                    i + 1, processSize[i], j + 1,
                    originalBlockSize[j], fragment);
                blockSize[j] -= processSize[i];
                allocated = 1;
                break;
            }
        }
        if(!allocated) {
            printf("Process %d of size %d is not allocated\n",
                i + 1, processSize[i]);
        }
    }
    return 0;
}
```

FIFO

```
#include <stdio.h>
int main() {
    int n, frames;
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);
    int ref[n];
    printf("Enter the reference string: \n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &ref[i]);
    }
    printf("Enter number of frames: \n");
    scanf("%d", &frames);
    int frame[frames];
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
    }
    int front = 0;
    int faults = 0;
    int hits = 0;
    for(int i = 0; i < n; i++) {
        int found = 0;
        for(int j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                found = 1;
                hits++;
                break;
            }
        }
        if(!found) {
            frame[front] = ref[i];
            front = (front + 1) % frames;
            faults++;
        }
    }
    printf("\nTotal Page Faults: %d\n", faults);
    printf("Total Page Hits: %d\n", hits);
    return 0;
}
```

Optimal

```
#include <stdio.h>
int main() {
    int n, frames;
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);
    int ref[n];
    printf("Enter the reference string: \n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &ref[i]);
    }
    printf("Enter number of frames: \n");
    scanf("%d", &frames);
    int frame[frames];
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
    }
    int faults = 0;
    int hits = 0;
    for(int i = 0; i < n; i++) {
        int found = 0;
        for(int j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                hits++;
                found = 1;
                break;
            }
        }
        if(!found) {
            int index = -1;
            for(int j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    index = j;
                    break;
                }
            }
            if(index == -1) {
                int farthest = -1;
                for(int j = 0; j < frames; j++) {
                    int k;
                    for(k = i + 1; k < n; k++) {
                        if(frame[j] == ref[k]) {
                            if(k > farthest) {
                                farthest = k;
                                index = j;
                            }
                        }
                    }
                    break;
                }
            }
            if(k == n) {
                index = j;
                break;
            }
        }
        faults++;
    }
}
```

```
        }
        frame[index] = ref[i];
        faults++;
    }
}
printf("\nTotal Page Faults: %d\n", faults);
printf("Total Page Hits: %d\n", hits);
return 0;
}
```

LRU

```
#include <stdio.h>
int main() {
    int n, frames;
    printf("Enter number of pages in reference string: \n");
    scanf("%d", &n);
    int ref[n];
    printf("Enter the reference string: \n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &ref[i]);
    }
    printf("Enter number of frames: \n");
    scanf("%d", &frames);
    int frame[frames];
    int time[frames];
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
    int faults = 0;
    int hits = 0;
    int counter = 0;
    for(int i = 0; i < n; i++) {
        int found = 0;
        for(int j = 0; j < frames; j++) {
            if(frame[j] == ref[i]) {
                hits++;
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(!found) {
            int lruIndex = 0;
            for(int j = 1; j < frames; j++) {
                if(time[j] < time[lruIndex]) {
                    lruIndex = j;
                }
            }
            counter++;
            frame[lruIndex] = ref[i];
            time[lruIndex] = counter;
            faults++;
        }
    }
    printf("\nTotal Page Faults: %d\n", faults);
    printf("Total Page Hits: %d\n", hits);
    return 0;
}
```